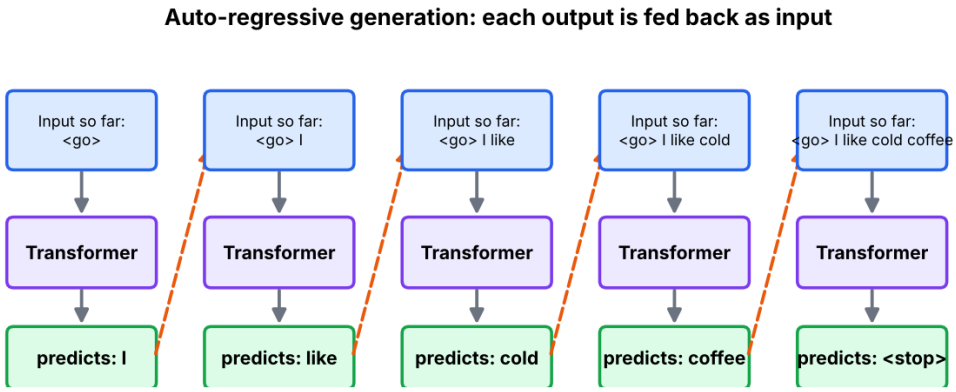
Week 4 Study Notes

Decoding Strategies & BERT

Lectures 17, 19, 20, 22, 18, 21, 24, 23, 25



The core idea of the whole week: a Transformer predicts ONE next word, and we feed it back in, again and again.

Made from your lecture transcripts. Each chapter has: one-line summary, simple explanation, pictures, a worked example, and a remember-box.

The big picture (read this first)

This week has two halves. Both are about the same Transformer, just used in two different ways.

Half	Lectures	The question it answers
1. Decoding (for GPT-style models)	17, 19, 20, 22, 18	The model gives me probabilities for the next word. **Which word do I actually pick?** Different rules (strategies) give different outputs.
2. BERT (encoder-only model)	21, 24, 23, 25	How do I build a model that understands text using words on **both sides**, and then how do I **use** it for real tasks?

Roadmap in the order of your syllabus

Lecture	Topic	One-line takeaway
17	Decoding overview	Decoding = choosing the next token. Deterministic vs stochastic.
19	Exhaustive search	Score ALL V^L sequences. Perfect but impossible.
20	Greedy & beam search	Greedy = best word each step. Beam = keep K best paths.
22	Top-K sampling	Keep K best words, then roll a dice among them.
18	Top-P (nucleus) + temperature	Keep the smallest set of words that adds up to probability P.
21	BERT & masked language modelling	Hide words, predict them from both sides.
24	Next sentence prediction (NSP)	Second BERT task: are these two sentences neighbours?
23	Pre-training & parameters	Data, layers, sizes, and counting ~110M parameters.
25	Adapting BERT to tasks	Feature extraction vs fine-tuning; question answering.

Tip for reading these notes

Colour code in every picture: **blue** = input/tokens, **green** = chosen/kept/good, **grey** = thrown away, **red/orange** = key warning or key step, **purple** = the Transformer/BERT model itself.

Note on the order

Your syllabus order is 17, 19, 20, 22, 18. That is actually a nice order: exhaustive, greedy/beam, top-K, then top-P (top-P fixes the problem of top-K, so it makes sense after it). Likewise 21, 24, 23, 25 follows BERT's story: masked LM, NSP, size, usage.

LECTURE 17 | DECODING

Decoding strategies: the big idea

In one line

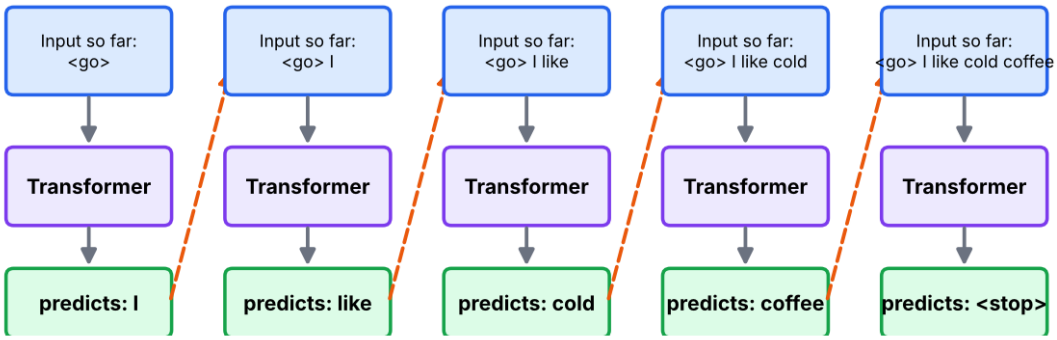
A trained Transformer only gives **probabilities for the next word**. **Decoding** is the rule you use to turn those probabilities into actual text.

1. What is the model really doing?

During training, the model saw text and learned to guess the **next token**. At generation time we repeat that guess again and again:

- You give a prompt, e.g. *I like*.
- The model outputs a probability for every word in the vocabulary.
- You pick one word, add it to the input, and run the model again.
- Stop when the model produces a special **end-of-sequence** token.

Auto-regressive generation: each output is fed back as input



Auto-regressive generation. The orange dashed arrows show the model's own output becoming the next input.

This works for everything: a question is just the first K tokens, and the answer is whatever the model continues with. Summarising works the same way: the paragraph + 'summarise' is the prompt, the summary is the continuation.

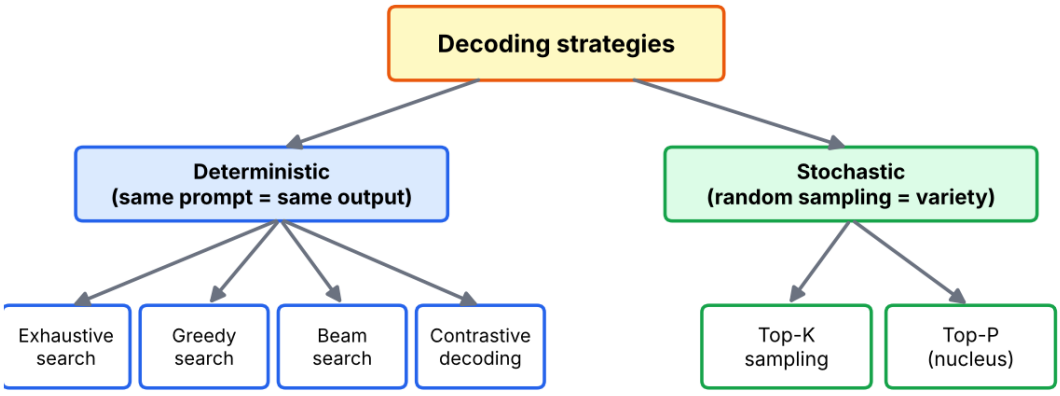
2. The problem: boring or repetitive output

Every computation inside the network is just addition and multiplication, so it is **deterministic**. If you always pick the most likely word, then *I like* will give the SAME continuation every single time. We want variety and creativity, without nonsense.

Example

Prompt: *Write a story starting with 'Once upon a time there was a king'*. If you always take the top word you get the same story forever. A human would add surprises (a weird imaginary country!). We want the model to be able to do that too.

3. The family tree of strategies



All decoding strategies in this week's lectures.

Type	Meaning	Strategies
Deterministic	Same input gives same output every time	Exhaustive, Greedy, Beam (and contrastive decoding)
Stochastic	Random sampling, so output can change each run	Top-K, Top-P (nucleus)

Remember

Transformer output = a probability list. **Decoding strategy = how you choose from that list.** The Transformer itself stays deterministic; randomness is added AFTER it, in the choosing step.

Exhaustive search (brute force)

In one line

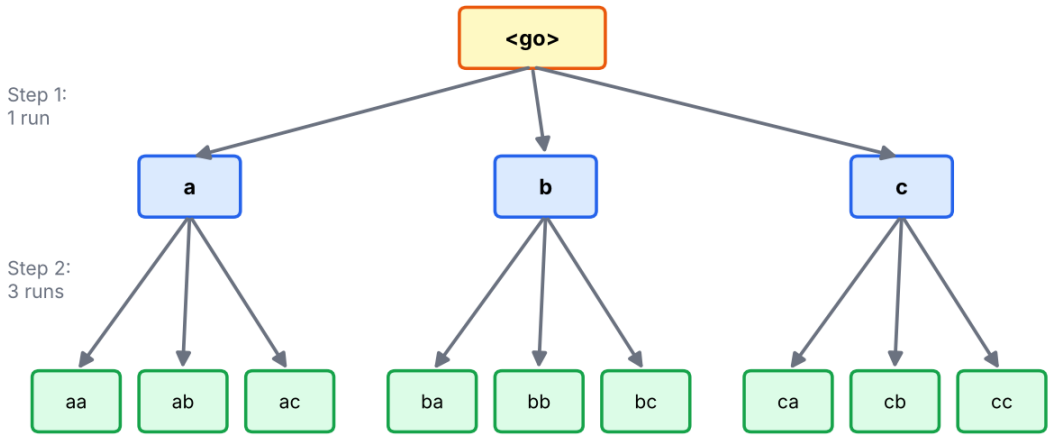
Generate **every possible** sequence, compute each one's probability, and return the best. Perfect answer, but impossible in practice.

1. How many sequences are there?

Vocabulary size = V. For a sequence of length L, each position can be any of V words, so there are $\mathbf{V^L}$ sequences.

Example

Real GPT vocabulary = 40,000 and length 5: $40{,}000^5$ = about 1 x 10^{23} sequences. Totally impossible. Toy example: V = 3 (a, b, c), L = 2 gives 3 x 3 = 9 sequences; L = 3 would give 27.



Exhaustive search tree for V=3, L=2. Step 1 runs the Transformer once, step 2 runs it 3 times (once per first word), step 3 would run it 9 times...

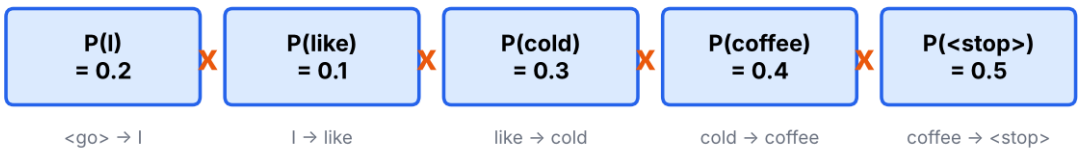
2. How do we score ONE sequence? (chain rule)

Probability of a sequence = multiply the probability of each word given all the words before it:

P(x1, x2, ..., xn) = P(x1) x P(x2 | x1) x P(x3 | x1,x2) x ... x P(xn | x1..xn-1)

Each factor comes from one run of the Transformer. Example from the lecture, the sequence **I like cold coffee <stop>** (numbers are made up for illustration):

P(I like cold coffee <stop>) = 0.2 × 0.1 × 0.3 × 0.4 × 0.5 = 0.0012



Chain rule: run the Transformer once per word, read off the probability of the word you wanted, multiply everything.

- Run 1: input gives a distribution. Read P(I) = 0.2.
- Run 2: input <go> I gives a new distribution. Read P(like | I) = 0.1.
- Run 3: input I like gives P(cold | I like) = 0.3. Run 4: P(coffee | I like cold) = 0.4. Run 5: P(stop | ...) = 0.5.
- Multiply: 0.2 x 0.1 x 0.3 x 0.4 x 0.5 = **0.0012**.

Why can't you reuse distributions between sequences? Because they are **conditional**: the distribution after 'I like' is different from the distribution after 'coffee like'. Every different prefix needs its own Transformer run.

3. Final step and why it fails

- Do this for all V^L sequences, sort by probability, output the top one.
- Deterministic: same input gives the same winner every time, so no creativity.
- Too slow to ever use. But it is the **gold standard** we try to approximate with cheaper methods.

Remember

Exhaustive search = best possible sequence in terms of probability, cost = V^L Transformer calls. Everything in lectures 20 to 18 is a trick to avoid this cost.

LECTURE 20 | DECODING

Greedy search & beam search

In one line

Greedy picks the single best word at every step. **Beam search** keeps the K best partial sentences alive at every step. Both are deterministic.

1. Greedy search

- Step 1: feed <go>, take the highest-probability word: **I**.
- Step 2: feed <go> I, take the best next word: **like**. Then **cold**, then **coffee**, then **<stop>**.
- Only ONE sequence is ever decoded, so it is very fast.

Problems with greedy:

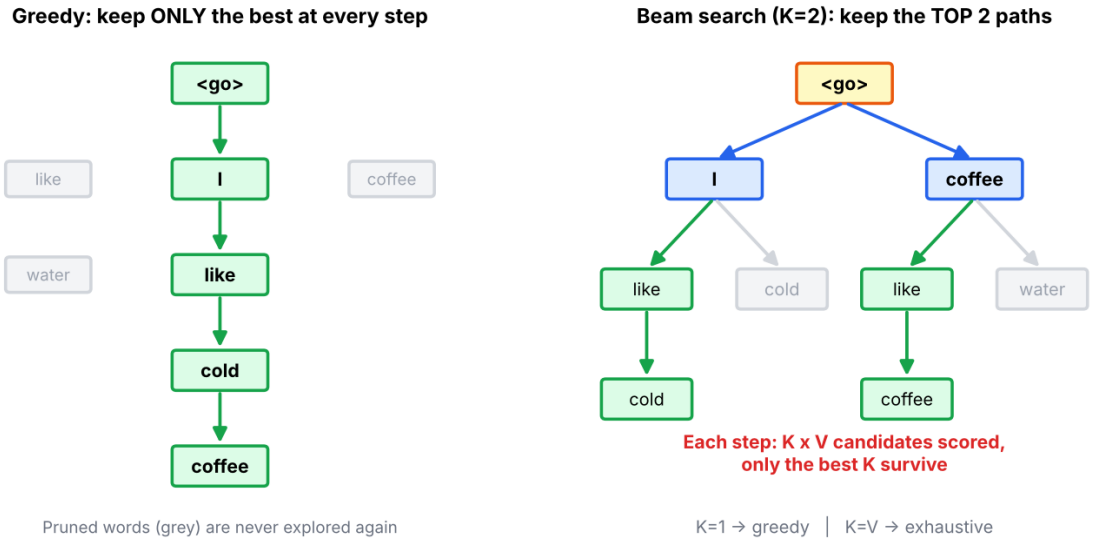
- **Same output every time** for the same input (no variety).
- **May not even be the most probable sequence.** Taking the best word now can lead you into a worse path later.

Example (from the lecture)

Greedy picked *I* first and ended with *I like cold coffee*. But if the 2nd-best first word, **coffee**, had been followed, the sequence *coffee like cold water* would have had a HIGHER total probability in the lecture's toy numbers. Greedy never found it because it threw *coffee* away at step 1. (To a human the first still sounds nicer; the point is only about probability. In a real vocabulary such missed better paths are common.)

2. Beam search

Fix for greedy: do not commit to one word. Keep a **beam of size K** (K candidate paths).



Left: greedy follows a single path. Right: beam search with K=2 keeps two paths alive and prunes the rest each step.

How one step of beam search works (K = 2):

- Step 1: feed <go>. Keep the top 2 words, e.g. **I** and **coffee**.
- Step 2: extend BOTH. That gives K x V candidates (2 x 6 = 12 in the lecture's example). Score them all; keep only the best 2 sequences.
- Step 3: repeat. Again K x V candidates, keep the best K.

- At the end, **backtrack** from the best final word to read the full sentence (e.g. stop <- costly <- is <- oil <- engine <- train <- a gives *a train engine oil is costly*).

K (beam size)	What you get	Cost
K = 1	Greedy search	1 path
1 < K < V	Beam search (a good approximation)	K x V scores per step
K = V	Exhaustive search	everything

Length normalisation: longer sequences multiply more numbers below 1, so they look less likely. At the end, divide the score by the length so short sentences do not win unfairly.

3. When is beam search a good idea?

Task	Beam search?	Why
Translation, summarisation	Yes	You want ONE correct, stable answer, not variety
Stories, jokes, poems, essays	No	You want surprise. Beam search always produces high-probability, repetitive, predictable text

Latency: greedy (fastest) < beam (K times more work) < exhaustive (huge).

Remember

Greedy and beam are **deterministic** and tend to be **repetitive/degenerate**. To get creativity we need **randomness**: sampling methods, next.

Top-K sampling

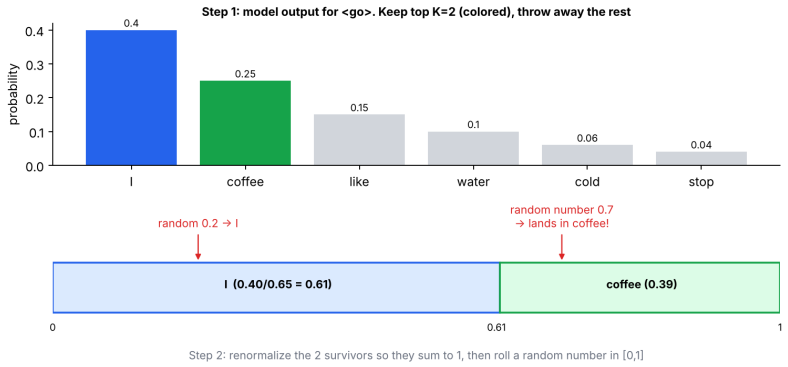
In one line

Keep only the K most likely words, **renormalise** their probabilities so they sum to 1, then **randomly sample** one.

Careful: this is NOT beam search with K paths. Beam search always keeps the best paths. Top-K sampling keeps only ONE path, but chooses each next word randomly among the K best.

Worked example (K = 2)

Model output for <go>: I = 0.40, coffee = 0.25, like = 0.15, water = 0.10, cold = 0.06, stop = 0.04.



Top: keep the 2 best tokens. Bottom: renormalise and sample with a random number.

- **Step 1:** keep the top 2: I (0.40) and coffee (0.25).
- **Step 2: renormalise.** Total = 0.65. So I = 0.40/0.65 = **0.61**, coffee = 0.25/0.65 = **0.39**.
- **Step 3: sample.** Draw a random number between 0 and 1. If it is below 0.61 pick **I**, otherwise pick **coffee**. Draw 0.7 gives **coffee** even though it is less likely!
- Feed the chosen word back in and repeat at the next step.

Think of it as a number line from 0 to 1 cut into slices. Each word gets a slice as wide as its probability. A random number picks a slice. Bigger slices are chosen more often, but small slices still get a chance.

Second step example

After I, suppose the top 2 are like = 0.55 and stop = 0.15. Normalised: like = 0.55/0.70 = **0.79** (lecturer said about 0.77), stop = **0.21**. Random number 0.9 gives **stop** and the sentence ends. Random number 0.5 gives **like** and the sentence goes on. Same prompt, different results.

Why is it OK to be random?

- You are NOT sampling from all 40,000 words. Only from the K words the model already considers good. It is **controlled randomness**.
- This allows low-probability (surprising) words to show up, which is where human-like creativity comes from.
- K = 1 is the same as greedy. Bigger K gives more variety but more risk.

Remember

Top-K = **filter to K best, renormalise, roll the dice**. Beam search = deterministic. Top-K = random. One catch (next lecture): a fixed K is not always the right K.

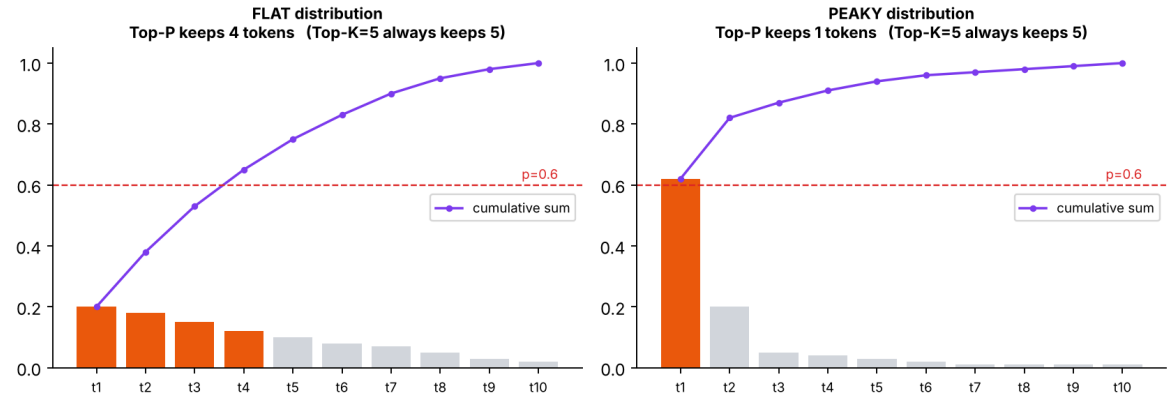
Top-P (nucleus) sampling & temperature

In one line

Instead of a fixed number of words (K), keep the smallest group of top words whose probabilities **add up to P** (like 0.6). The number of words changes automatically.

1. The problem with a fixed K

- **Flat distribution** (many words about equally likely), e.g. after *She said I never ...*: K = 5 is fine, but we might also be wrongly cutting words that are as good as the 5th.
- **Peaky distribution** (one word dominates), e.g. *I ate the pizza while it was still ...* (hot gets almost all the probability): K = 5 includes weak words like random junk, which can produce nonsense.



Same threshold p = 0.6. Flat distribution keeps many tokens, peaky distribution keeps only 1 or 2. Orange = kept, grey = dropped, purple line = running total.

2. Top-P step by step

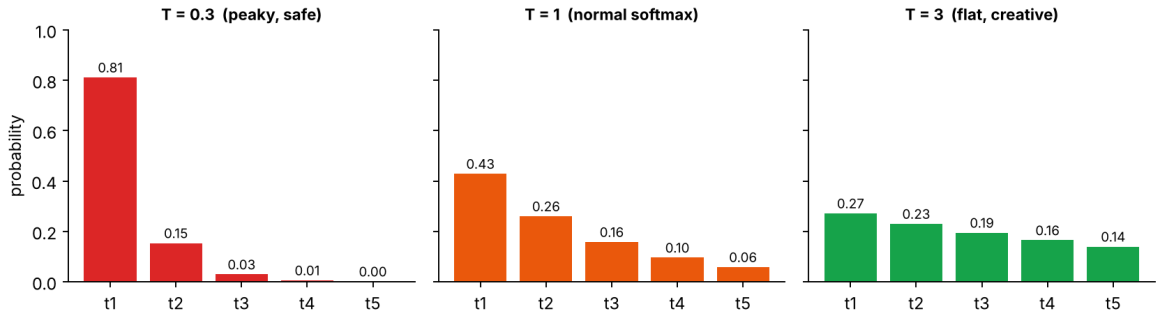
- Sort the probabilities from highest to lowest.
- Choose P between 0 and 1, e.g. P = 0.6.
- Add up probabilities from the top. As soon as the sum reaches P, draw the line there. Those tokens are your **nucleus**.
- Renormalise them so they sum to 1 and sample, exactly like top-K.

Example

Pizza sentence (peaky): hot = 0.55, cooling = 0.20... Running sum: 0.55, 0.75. Reached 0.6 after 2 tokens, so the nucleus is {hot, cooling}. Renormalise (the lecture example uses hot = 0 to 0.8, cooling = 0.8 to 1), pick a random number, done.
She said I never (flat): the first token alone is only about 0.1, so you need around 5 or 6 tokens before reaching 0.6, and all of them are allowed.

3. Temperature: another way to control randomness

Softmax with temperature T: probability of token i = $\exp(u_i / T)$ / sum of $\exp(u_j / T)$, where u are the model's raw scores (logits). T = 1 is the normal softmax.



Same scores, three temperatures. Low T sharpens (safe), high T flattens (creative).

Temperature	Effect on distribution	Output
T < 1 (e.g. 0.1)	Divides by a small number, so big scores get much bigger: **peaky**	Safe, repetitive
T = 1	Normal softmax	Default
T > 1	**Flatter**, many words almost equally likely	More creative, more risk

The lecture says top-P is the **better** fix than temperature for the K problem, because it adapts to each distribution.

Remember

Top-K: fixed number of words. Top-P: fixed amount of **probability mass**, so the word count adapts (flat gives more words, peaky gives fewer). Deterministic strategies (exhaustive, greedy, beam) vs stochastic (top-K, top-P) is the full picture of decoding. Next: BERT, an encoder-only model.

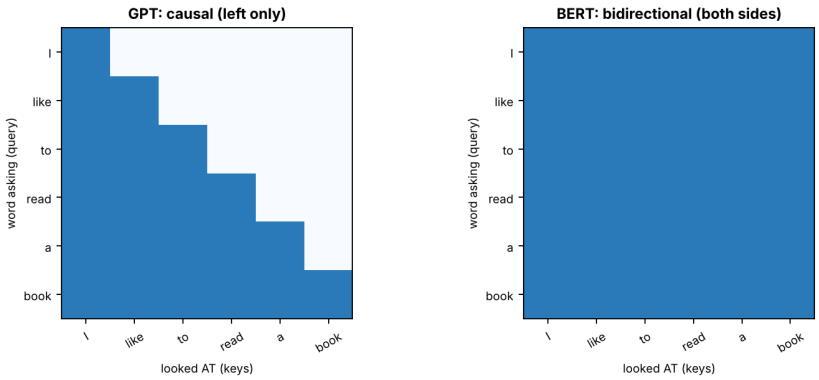
BERT & masked language modelling

In one line

BERT (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers) hides some words and learns to predict them using words on **both the left AND the right**.

1. GPT vs BERT: why do we need both directions?

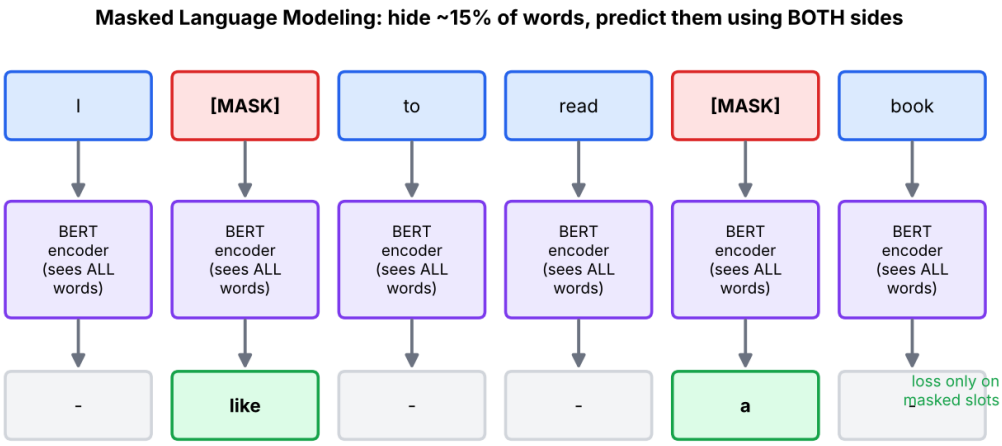
- **GPT** (decoder-only) uses *causal language modelling*: predict the next word looking only at the past (left). Natural for text generation, and most modern LLMs work this way.
- But some tasks need the future context too. Named entity recognition: *I went from Delhi to Goa*. The words 'to Goa' help us know 'Delhi' is a location. Fill in the blank: ____ *has shipped its new OS*: the words after the blank tell us it is a company.



Attention maps. Dark = allowed to look. GPT only looks at itself and earlier words. BERT looks at every word.

2. Masked Language Modelling (MLM)

Idea: take a sentence, **hide ~15%** of the words, and ask the model to guess the hidden ones. It is like a fill-in-the-blank exam.



The encoder sees the whole sentence with [MASK] in the blanks and predicts the missing words.

- Replace chosen words by a special token **[MASK]** (it is just another word in the vocabulary).
- Feed the sentence to the Transformer **encoder** (no cross-attention, no causal mask). Each of the T positions gets an output vector.
- At the masked positions, predict a probability over the vocabulary. Loss = cross-entropy (maximise the log-probability of the true word).

- Labels come for free: the text came from Wikipedia, so you know the hidden word. No human labelling.

Why not just 'mask the attention' like GPT does?

GPT hides future words by zeroing their attention. If we did that for BERT's blanks, the words would vanish from the sentence. But the model must **know there is a blank** and fill it. So we use a visible [MASK] token instead. Think of it as blacking out part of an image and asking the model to repaint it.

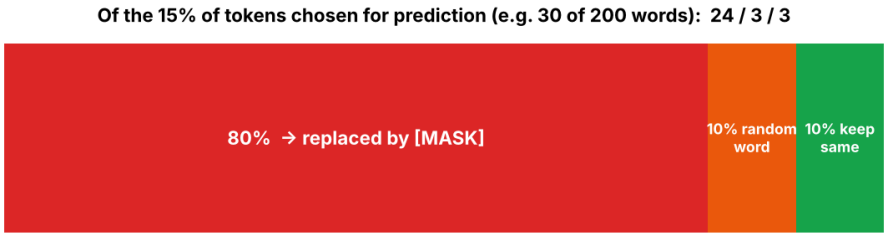
The masked words are predicted **independently**, all at once, using only the unmasked words. It does not predict the 2nd blank first and reuse it for the 8th blank. This makes the task harder, so the model must really learn how words relate.

3. Why 15%?

- Mask too much (say 85%): too little context, the task is nearly impossible, loss stays high.
- Mask too little (1 of 20 words): too easy, little is learned.
- Empirically, **15%** worked well (later papers study other values).

4. The 80 / 10 / 10 trick

Of the tokens chosen for prediction, BERT does NOT always use [MASK]:



Example: 200 tokens gives 30 chosen tokens (15%), split 24 / 3 / 3.

Share	What happens to the word	Example with 'cat'
80%	Replaced by [MASK]	The [MASK] sat on the mat
10%	Replaced by a random word	The **apple** sat on the mat (still predict 'cat')
10%	Kept unchanged	The **cat** sat on the mat (still predict 'cat')

Why? In real tasks (sentiment, QA, entailment) the input has **no [MASK] tokens at all**. If BERT only ever saw [MASK] in training, it would be confused by normal text. The 10% unchanged / random cases teach it to produce good representations for real words too.

Heads-up on the transcript

The lecturer says at one point that BERT 'uses causal language modelling'. That is a slip: BERT uses **masked** language modelling, GPT uses causal. Also, BERT-base has 12 layers and 12 heads; BERT-large has 24 layers and 16 heads (see Lecture 23).

Remember

BERT = encoder stack + MLM. Bidirectional. Hide 15% of words (80% [MASK], 10% random, 10% unchanged), predict them. Good for understanding tasks, not designed for free text generation.

Next sentence prediction (NSP) & BERT's input

In one line

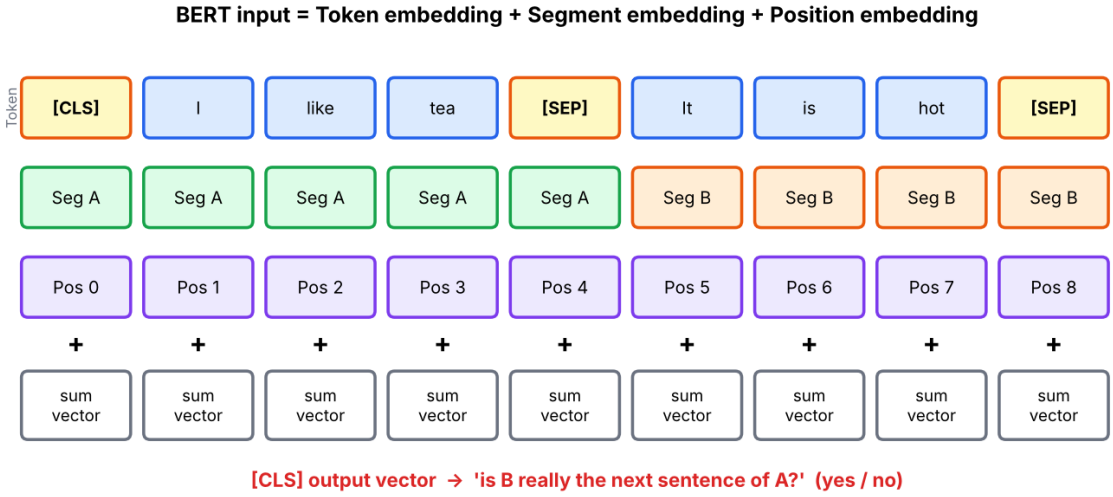
BERT's second training task: given two sentences A and B, predict whether B really came **right after** A. Also, BERT's input = token + segment + position embeddings.

1. Why NSP?

Some tasks involve **pairs** of texts: question + passage (QA), claim + sentence (entailment). MLM alone doesn't teach how two sentences relate, so BERT added a second objective.

2. How it works

- Input: **[CLS]** sentence A **[SEP]** sentence B **[SEP]**. [CLS] is a special start token, [SEP] separates the sentences. Length becomes T + 2 (or +3 with the final SEP).
- Take the output vector of [CLS]. [CLS] has attended to every word, so it acts like a summary of the pair.
- Put a small binary classifier on it: **IsNext (1)** or **NotNext (0)**. Loss = 2-class cross-entropy.



BERT's input is made of three embeddings added together; the [CLS] output feeds the NSP classifier.

3. Free training data again

Pair	Label	How to make it
A, B (B really follows A in Wikipedia)	1 (IsNext)	50% of the time, take consecutive sentences
A, random B from the corpus	0 (NotNext)	50% of the time, swap B for a random sentence

Example

A = 'I went to the market.' B = 'I bought fresh mangoes.' gives label 1.
A = 'I went to the market.' B = 'The Transformer has 12 layers.' gives label 0.

4. Total loss

Total loss = MLM loss + NSP loss. Both heads sit on top of the same encoder layers; the last layer gives vectors for each token (for MLM) and the [CLS] vector (for NSP).

5. Did NSP help?

It helped on two-sentence tasks at first. But within about a year, follow-up work showed NSP does **not** help much on downstream tasks, so it was usually **dropped**. Later BERT-style models use **only MLM**.

6. The three embeddings (BERT's input)

Embedding	Table size	What it tells BERT
Token	V x d_model (30,000 x 768)	Which word it is
Segment	2 x d_model	Is this word in sentence A or sentence B?
Position	512 x d_model	Where in the sequence is it?

For each word, look up its 3 vectors and **add them** into one d_model vector. That sum is what enters the encoder.

Remember

[CLS] = summary slot, [SEP] = boundary. Input = token + segment + position. Training = MLM + NSP (NSP later abandoned).

Pre-training details & counting parameters

In one line

What BERT was trained on, how big it is, and how to count its ~110 million parameters yourself.

1. Data and settings

Item	Value
Training data	BookCorpus (800M words) + English Wikipedia (2.5B words)
Vocabulary	30,000 (actual: 30,522) sub-word pieces
Max context length	512 tokens
Training tasks	MLM + NSP

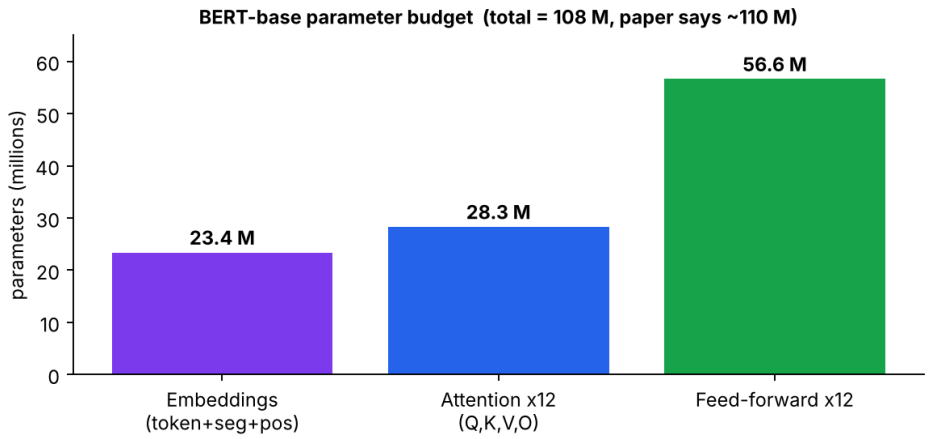
(By modern standards these datasets are small; today's LLMs use far more.)

2. Two sizes

	BERT-base	BERT-large
Layers	12	24
Hidden size d_model	768	1024
Attention heads	12	16
Feed-forward size (4 x d_model)	3072	4096
Parameters	~110 M	~340 M

Once you know the Transformer, these few numbers (layers, d_model, heads, FFN size, vocab) fully describe the network.

3. Counting parameters of BERT-base (biases ignored)



Where BERT-base's parameters live.

Part	Calculation	Parameters
Token embeddings	30,000 x 768	23.04 M
Segment embeddings	2 x 768	~0.002 M

Part	Calculation	Parameters
Position embeddings	512 x 768	0.39 M
Embedding total		**~23.4 M**
Attention Q, K, V (one layer)	12 heads x 3 matrices x (768 x 64) = 3 x 768 x 768	1.77 M
Output projection W_O	768 x 768	0.59 M
Feed-forward	768 x 3072 + 3072 x 768	4.72 M
One layer		**~7.1 M**
12 layers	12 x 7.1 M	**~85 M**
Total	23.4 M + 85 M	**~108 M**

The lecture reaches about 107 M; the paper says ~110 M. The small gap is biases, LayerNorm weights, and using 30,000 instead of the real 30,522.

Picture of one head

Each of the 12 heads turns a 768-d vector into a 64-d vector (3 matrices of 768 x 64). 12 heads x 64 = 768, so concatenating them gives back 768 numbers, and W_O (768 x 768) mixes them.

Remember

BERT-base: 12 layers, 768 dim, 12 heads, FFN 3072, vocab 30k, ~110M parameters. Embeddings are about 23M, the 12 layers carry the rest (~85M). GPT-1 was in the same ballpark (~117M).

Adapting BERT to downstream tasks

In one line

Pre-training teaches BERT language in general. To make it do a specific job (sentiment, QA), we use a small amount of labelled data in one of two ways: **feature extraction** or **fine-tuning**.

1. Pre-training vs downstream

- **Pre-training**: huge amount of *unlabelled* text (Wikipedia + books), tasks are MLM/NSP, labels are free. BERT learns how words interact. It is NOT trained for sentiment or QA.
- **Downstream task**: small amount of *labelled* data (expensive). Hope: since BERT already understands language, a little data is enough.

> **Analogy**
>
> A teenager has learned language for years. Show them a few examples of positive and negative reviews and they can quickly label new ones. Pre-trained BERT is the teenager; the labelled data is the few examples.

2. Approach A: BERT as a feature extractor



Left: BERT frozen, only a small classifier is trained. Right: fine-tuning, everything is trained.

- Pass each review through the **frozen** BERT (no masks now).
- Take the **[CLS]** vector from the last layer: 768 numbers that summarise the review.
- You now have a normal ML dataset: X = 768 features, y = label (0 or 1).
- Train any classifier on it: logistic regression, k-NN, a small feed-forward network ('classification head').
- BERT's weights are never changed. Extract features once, then forget BERT.

3. Approach B: fine-tuning

- Put a new layer on [CLS]: 768 x 2 weights followed by a softmax over {positive, negative}.
- Loss = -log(probability of the correct class).
- **Backpropagate through the whole network**: the new head AND all BERT weights (WQ, WK, WV, ...) adjust to the task.
- Result: BERT specialises for the task. It converges quickly even with few labelled examples.

	Feature extraction	Fine-tuning
BERT weights	Frozen	Updated
What is trained	Only a small classifier	Head + all of BERT
Cost	Cheap, done once	More compute
Performance	Good	Usually better, needs fewer labelled examples

Why are there no [MASK] tokens at test time?

A review with random blanks would be nonsense. That is exactly why BERT's pre-training kept 10% of the chosen words unchanged (Lecture 21).

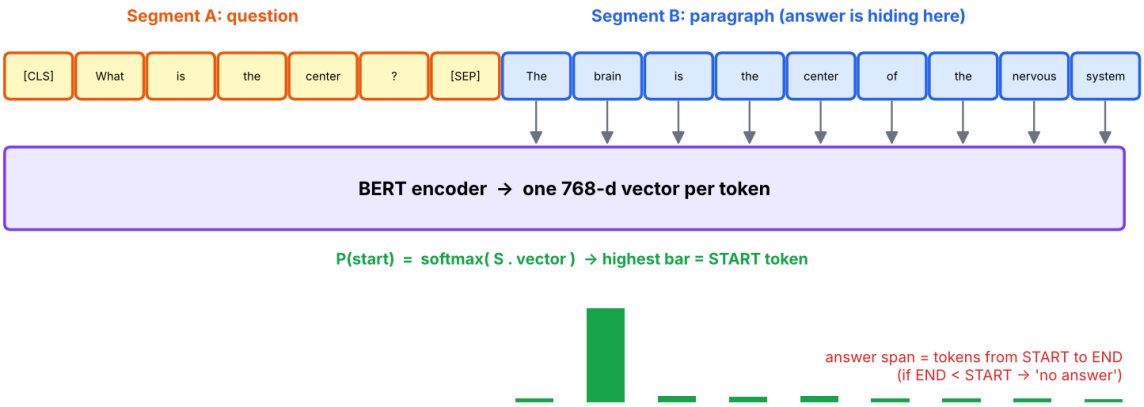
A note on data: if the training text has no Indian mythology, the model cannot know about it. Diverse pre-training data (or extra fine-tuning data) matters.

4. Extractive question answering (SQuAD style)

Given a question and a paragraph, the answer is a **span of words copied from the paragraph**.

Example

Short case: Paragraph *The brain is the center of the nervous system.* Question *What is the center?* Answer = **brain** (a span copied from the paragraph, not a rewrite).
Longer case from the lecture: a paragraph about a space mission; the answer span starts at the token 'role' and ends at 'spacecraft' (the role of artificial intelligence in guiding the spacecraft).



QA with BERT: input = question (segment A) + paragraph (segment B). We score each paragraph token as START and as END.

- Input: [CLS] question [SEP] paragraph. Segment A = question, segment B = paragraph.
- BERT outputs a 768-d vector for each paragraph token.
- Add a new learnable vector **S** (start) and **E** (end), both 768-d.
- P(token i is START) = softmax over all tokens of ($S \cdot vector_i$). Same for END with E.
- Pick START = token with the highest start probability, END = token with the highest end probability.
- If END comes before START (j < i) the span is invalid, so answer = 'no answer' (empty string).

Example from the lecture

Highest start score is token 9 (the word 'role') and the highest end score is token 17. So the answer is everything from token 9 to token 17: 'role of artificial intelligence in guiding the spacecraft'.

Remember

Feature extraction = freeze BERT, train a small classifier on [CLS]. Fine-tuning = train everything. QA = learn two vectors S and E to pick the START and END token of the answer span.

Week 4 cheat sheet

Decoding strategies side by side

Strategy	What it does	Same output each run?	Creative?	Cost	Good for
Exhaustive	Scores all V^L sequences	Yes	No	Impossible	Theory only
Greedy	Best word each step	Yes	No	Lowest	Fast baseline
Beam (K)	Keeps K best paths	Yes	No	K x greedy	Translation, summarisation
Top-K	Sample among K best words	No	Yes	Low	Stories, chat
Top-P	Sample among words adding to P	No	Yes	Low	Stories, chat (adapts better)
Temperature	Reshapes the softmax (low = peaky, high = flat)	No (used with sampling)	Controls it	Free	Tuning randomness

GPT vs BERT

	GPT (decoder-only)	BERT (encoder-only)
Training task	Predict next word (causal LM)	Fill in masked words (MLM) + NSP
Sees	Only the left (past)	Both sides
Best at	Generating text	Understanding text: classification, QA, NER
Special tokens	start / end of sequence	[CLS], [SEP], [MASK]

Formulas to remember

- Number of sequences: V^L
- Chain rule: P(x1..xn) = P(x1) x P(x2|x1) x ... x P(xn|x1..xn-1)
- Beam search cost per step: K x V candidates
- Top-K renormalise: $p'_i = p_i$ / (sum of kept probabilities)
- Temperature softmax: $\exp(u_i/T)$ / sum $\exp(u_j/T)$
- BERT masking: 15% of tokens chosen; of those 80% [MASK], 10% random, 10% same
- BERT-base parameters: about 23M embeddings + 12 x 7M layers = ~110M
- QA: P(start=i) = softmax(S . h_i), P(end=j) = softmax(E . h_j), need j >= i

Quick self-test (answers below)

- 1. Why can't we use exhaustive search with a 40,000-word vocabulary?
- 2. What is the difference between top-K sampling and beam search with K paths?
- 3. Why does top-P adapt better than top-K?
- 4. What does a low temperature do?
- 5. Why does BERT sometimes leave a chosen word unchanged?
- 6. In fine-tuning vs feature extraction, which weights are updated?

Answers: (1) $40{,}000^{L}$ sequences, far too many Transformer runs. (2) Beam keeps K best paths deterministically; top-K keeps ONE path and randomly samples each word among the K best. (3) It chooses the number of words from the shape of the distribution: flat gives more words, peaky gives fewer. (4) Makes the distribution peaky: safer, less variety. (5) Real task inputs have no [MASK], so it must handle normal words. (6) Feature extraction: only the small head. Fine-tuning: head and all of BERT.